The Judgment Ledger — UI Generation Prompt (Figma Make / Supabase)

Build a web application called "The Judgment Ledger" with three connected screens, sharing one consistent dark, technical visual style: deep slate background, amber as the primary accent color for anything requiring attention or action, teal for "safe/trusted" states, red for "problem/blocked" states, monospace font for data and evidence, clean sans-serif for UI labels and body text.

Screen 1 — Submission screen. A form where an engineer submits a code change for deploy. Fields: service name, files changed, commit reference, author. Shows automated test pass/fail status. A "What Happens Next" panel explaining in three plain steps what the system will do (classify the change, check trust boundary, ship or escalate). A primary submit button that starts the workflow.

Screen 2 — Escalation review screen. Shown to a Senior Approver when a change needs human review. Two-column layout: left side shows the change context (diff summary, category, blast radius as a list of affected services). Right side shows a trust gauge — a visual bar showing how much evidence exists for this category against the threshold needed to auto-approve, plus a written explanation of why this specific change was escalated. Below that, an evidence trail: a vertical list of past similar changes, each tagged with a status color (safe, problem, still pending) and a one-line outcome description. At the bottom, three actions: Approve & Deploy, Request More Signal, Block Change. Include a visible open-timer note stating the change stays blocked until reviewed, with no automatic timeout approval.

Screen 3 — Weekly policy review screen (Engineering Leadership). A dashboard listing every category currently requiring human approval. For each category, show: current sample size vs. the minimum required, correction history with severity noted (not just count), and the system's recommendation — expand trust, restore trust, or hold as-is — with its reasoning written in plain language. Each recommendation needs an individual Approve / Reject action. Include a small note stating this is the only way trust boundaries can be loosened; nothing here is applied automatically.

Data behavior: All three screens should read from and write to a shared backend table representing categories and their trust status, and a table representing individual submissions with their decisions and outcomes. The escalation screen's decision should update the submission record and notify screen 3's data if a pattern of overrides emerges. The weekly review screen's approvals should be the only thing that changes a category's trust status upward; a separate background process (representing the Reconciliation Agent) should be able to tighten a category's status down automatically.

Preserve the core interaction principle across all three screens: nothing here should let a person approve something without seeing the actual evidence behind it first.